

Reviewer Pack — Minimal Order of p-adic Galois Representations Bounds Quantu...

Entry f54c017c6d3e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 16:31:20 UTC

Minimal Order of p-adic Galois Representations Bounds Quantum Circuit Depth

Entry ID: f54c017c6d3e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 16:31:20 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (p-adic Galois Representations) **Field B** (complexity object): Quantum Information Theory (Quantum Circuit Complexity)

Statement:

[‘For every satisfiable Boolean formula F represented by a quantum circuit of depth D , the minimal order of a p-adic Galois representation attached to its truth table is $O(p^D)$.’, ‘This bound holds for all prime numbers p and instances with $n \leq 40$ variables.’, ‘Equivalently, for a fixed prime p , if there exists an instance F such that the minimal order of the associated p-adic Galois representation is less than $O(p^D)$, then $D \leq \log_2(n)$.’]

Rationale (proposer’s reasoning):

[‘p-adic Galois representations are known to encode arithmetic properties of numbers, and their orders are connected to deep number-theoretic problems.’, ‘Quantum circuit depth has been studied in the context of quantum algorithms, and a direct connection to number theory could provide new insights into both fields.’, ‘This conjecture aims to exploit this potential connection by proposing a quantitative relation between the two.’]

Taxonomy category: p-adic_galois_representation_quantum_circuit_depth (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7b126533d627beef

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all generated Boolean formulas F with $n \leq 40$ variables and depth $D \leq \log_2(n)$, the minimal order of the p-adic Galois representation attached to its truth table is $O(p^D)$ with at least 80% of seeds having a metric mean of the minimal orders $\leq D$. The conjecture is falsified if any seed produces a case where the minimal order is less than $O(p^D)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def truth_table(formula):
        if formula == '0' or formula == '1':
            return [int(formula)]
        elif formula.startswith('not'):
            subformula = formula[4:-1]
            return [1 - val for val in truth_table(subformula)]
        elif formula.startswith('and'):
            left, right = formula[4:-1].split()
            return [val1 and val2 for val1 in truth_table(left) for val2 in truth_table(right)]
        elif formula.startswith('or'):
            left, right = formula[3:-1].split()
            return [val1 or val2 for val1 in truth_table(left) for val2 in truth_table(right)]

    def min_p_adic_order(truth_values):
        order = 0
        for value in truth_values:
            if value == 1:
                order += 1
        return order

    n = random.randint(5, 40)
    D = random.randint(2, int(n.bit_length()))

    # Generate a random Boolean formula of depth D and n variables
    def generate_formula(depth):
        if depth == 0:
            return str(random.choice([0, 1]))
        else:
```

```

        op = random.choice(['and', 'or'])
        left = generate_formula(depth - 1)
        right = generate_formula(depth - 1)
        return f"({op} {left} {right})"

formula = generate_formula(D)

truth_values = truth_table(formula)
min_order = min_p_adic_order(truth_values)

conjecture_holds = min_order <= D
counterexample = "" if conjecture_holds else f"Formula: {formula}, Min Order: {min_order}, Exp

return {
    "metric_name": "Minimal p-adic Order",
    "metric_value": min_order,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_order = sum(res["metric_value"] for res in results) / len(results)
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_order} std=NA support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"First failing seed\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5a45234f.py", line 76, in <module>

```

    result = run_trial(seed)
    ^^^^^^^^^^^^^^^^^

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5a45234f.py", line 57, in run_trial

```

    min_order = min_p_adic_order(truth_values)

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5a45234f.py", line 36, in min_p_adic_or
    for value in truth_values:
TypeError: 'NoneType' object is not iterable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test ensuring that all necessary data is collected and no errors occur during execution.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17576
2	propose	ollama_remote	glm4:latest	0	0	10140
3	propose	ollama_remote	glm4:latest	0	0	10732
4	preregistration	ollama_remote	glm4:latest	0	0	5936
5	novelty	ollama_remote	glm4:latest	0	0	4757
6	novelty	ollama_remote	glm4:latest	0	0	5131
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14633
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10031
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10454
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8416
11	judge	ollama_remote	glm4:latest	0	0	10206

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 108010 ms total latency. Provider mix: {'ollama remote': 11}

(full prompt+response transcripts available in [research/audit/f54c017c6d3e.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f54c017c6d3e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp sandbox/test *.py (per-cycle)

Replay tarball: `research/replay/f54c017c6d3e.tar.gz` (if generated)